

Side Streets: A Local Places Guide

Project description

Imagine a friend visiting your city for the first time. They ask where to get coffee, find a small bookshop, or spend an afternoon outdoors. You have recommendations, but they are scattered across messages, saved links, and memory. In this project, you will build **Side Streets**, an application that brings those recommendations together and helps people find local places to visit.

Each recommendation contains a name, category, neighborhood, address, description, and a tip for visitors. People using the application can browse and search for recommendations, save places to a shared shortlist, and add, edit, or remove recommendations to keep the collection up to date.

Business requirements

1. Browse places

Users can view the collection and open a specific place to read its information.

2. Search and filter

Users can search by name, neighborhood, or description. They can filter by category and use both filters together.

3. Add a place

Users can submit a recommendation with all required information. The application rejects submissions that do not meet the specified rules.

4. Edit a place

Users can replace a place's information while keeping its identity and shortlist membership.

5. Delete a place

Users can remove a recommendation. Its shortlist entry, if present, is also removed.

6. Use a shared shortlist

Users can save places, view saved places, and remove places from the shortlist. Unsaving does not delete the recommendation. Saving twice does not create duplicate entries.

7. Keep saved information

Recommendations, edits, deletions, and shortlist membership remain after the application restarts.

8. Use a browser interface - Frontend

A browser page supports browsing, searching, filtering, managing recommendations, and the shortlist. This requirement is considered lowest priority. The backend is the main focus.

Technical requirements

Use Python 3.10 or newer, FastAPI 0.115.12, Uvicorn 0.34.2, Pydantic, and Python's sqlite3 module. Use pytest 8.3.5 and httpx 0.28.1 for tests. Store data in data/guide.db. Use SQL statements through sqlite3 for database operations.

Place data

Field	Required value
name	String, 2–80 characters
category	Coffee & bites; Art & culture; Outdoors; Little shops
neighborhood	String, 2–60 characters
address	String, 3–120 characters
description	String, 20–1000 characters
tip	String, 5–200 characters

Strip surrounding whitespace before checking text lengths. Reject extra fields. The server assigns an **integer id**.

Responses include shortlisted as a JSON boolean. POST and PUT require all six editable fields; neither accepts id or shortlisted in its content body.

API operations

An endpoint is an HTTP method and URL path. Return JSON except for successful DELETE, which has no body.

Method and path	Success
GET /api/health	200
GET /api/places	200
GET /api/places/{place_id}	200
POST /api/places	201
PUT /api/places/{place_id}	200
DELETE /api/places/{place_id}	204
GET /api/shortlist	200
PUT /api/places/{place_id}/shortlist	200

Missing places: 404 with detail "This place does not exist." Invalid requests: 422. Shortlist input: {"shortlisted": true} or {"shortlisted": false}; reject strings, numbers, and extra fields.

Search, storage, and tests

Users can narrow the list of places by entering search text, selecting a category, or using both. The search parameter, named q, accepts up to 100 characters and looks for that text in each place's name, neighborhood, or description. The category parameter accepts one of the four categories listed above. When both are provided, return only places that match both. When neither is provided, return all places. List the results by ID, from lowest to highest. The following pages explain how to check this behavior.

When the application starts, create any required database tables that do not already exist. Add the initial example places only once. If users later delete those places, restarting the application must not add them again. Store shortlist entries in a separate table. Each entry must refer to an existing place, and each place can appear in the shortlist only once. Deleting a place must automatically remove its shortlist entry ("cascade deletion"). Organize the application files as described on page 4, and run automated tests using temporary databases so they do not change the application's stored data.

Frontend

Serve the browser page at / and its files under /static. Use the API for the page 1 operations. Implementation details pending backend completion.

Setup and how to run the checks

Work from the folder containing app/ and requirements.txt. Create the required files before starting the server. The following commands use Windows PowerShell. No virtual-environment activation is needed.

Terminal A: install and start

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

Keep terminal A running. Stop with Ctrl+C. Restart using the same Uvicorn command. Use terminal B, in the same folder, for requests and scripts.

FastAPI /docs first

1. Open <http://127.0.0.1:8000/docs> in your browser.
2. Expand an endpoint, click Try it out, enter the parameters or JSON body, then click Execute.
3. Read Server response for the actual status and body. The example responses below it are documentation.
4. Execute GET /api/health. Expect 200 and {"status":"ok"}.

Terminal B: equivalent health check

```
$base = 'http://127.0.0.1:8000'
curl.exe -i "$base/api/health"
```

Run these commands in PowerShell. The first stores the server's address in a variable named \$base. The second sends a request to /api/health. The -i option displays the response headers, including the HTTP status code, followed by the response body. Expect status 200 OK and {"status":"ok"}.

Check sequence

Run the checks on pages 5-11 in order with the supplied seed data. Choose the browser sequence or the terminal sequence for each exercise. Running both repeats changes. If switching to the terminal, define \$base and the page 6 body, and create a new example to obtain \$placelId. Browser requests do not set PowerShell variables.

Never assume a new record has a particular ID. Use the returned ID. Use your own example for edits and deletions, keeping the three seed records available for search checks. Restarting and running scripts or tests require a terminal, not /docs.

[FastAPI: first steps and interactive API documentation](#)

Required files and starting data

File	Responsibility
requirements.txt	The four pinned dependencies listed on page 2
app/__init__.py	Python package marker
app/main.py	FastAPI application, startup, router, frontend serving
app/models.py	Category, PlaceInput, Place, ShortlistInput
app/routes.py	API endpoints and HTTP error handling
app/db.py	Connections, initialization, SQL reads and writes
app/schema.sql	Tables: places, shortlist, app_metadata
app/seed.json	The three initial place records
scripts/	SQLite learning, inspection, and relationship scripts
tests/conftest.py	Temporary database and client fixtures
tests/test_api.py	Backend behavior tests
static/	index.html, app.js, styles.css

Store the place records in a table named places. Each place must have an id that identifies it.

Store saved selections in a separate table named shortlist. Each row contains a place_id pointing to a place in the places table. A place can appear in the shortlist only once. The referenced place must exist, and deleting it must automatically remove its shortlist entry.

Use a primary key on shortlist.place_id to prevent duplicates. Use a foreign key referencing places.id to enforce the relationship. Add ON DELETE CASCADE so SQLite removes the related shortlist entry when a place is deleted. Enable foreign-key checks whenever you open a database connection.

Apply the field-length limits and allowed categories in the database table definition as well as in the input model. This ensures that data written directly by a script must also follow those rules.

When writing SQL, use ? placeholders for values and pass those values separately. Do not insert user input directly into the SQL text.

Save successful database changes by committing them. If an operation fails, roll back its pending changes. Close the connection when the work finishes.

Before the application accepts requests, create any required tables that do not exist. Use the app_metadata table to record whether the initial example data has already been added. Add the examples and this record in one transaction, so they are either saved together or not saved at all. Restarting the application must not add the examples again.

Supplied seed file

The accompanying seed.json contains The Sunday Room (Coffee & bites, Old Town), Saltwater Steps (Outdoors, Harbour), and Ink & Paper (Little shops, Old Town). Copy it into app/seed.json. On a new database, initialization inserts these three places.

Supporting scripts

Provide scripts/__init__.py, create_learning_database.py, insert_learning_place.py, read_learning_places.py, inspect_guide_database.py, and check_shortlist.py. The first three use data/learning.db for the SQLite exercise. The application uses data/guide.db; these are separate files.

```
.\env\Scripts\python.exe -m scripts.create_learning_database
.\env\Scripts\python.exe -m scripts.insert_learning_place
.\env\Scripts\python.exe -m scripts.read_learning_places
```

On a fresh learning database, expect one inserted place and the same record when reading again. Repeating the insert adds another row.

[Python: SQLite connections, placeholders, and transactions](#)

Check 1: Browse, retrieve, and search

Expected behavior

GET /api/places returns an array ordered by id. Every place has its six content fields, id, and shortlisted. A new seed record is not shortlisted. GET by ID returns one object or 404.

Trim surrounding whitespace from q. Search for a substring across name, neighborhood, and description with spaces between the fields. Use the taught SQLite lower() and instr() approach: ASCII letter case does not affect matching. Category matching is exact. Empty or whitespace-only q adds no text restriction. No matches returns 200 with [].

FastAPI /docs first

1. GET /api/places without filters. On a fresh database, expect the three supplied records.
2. GET /api/places/{place_id} with an ID from that response. Expect 200 and the matching record.
3. List with q=old and category=Little shops. Expect only Ink & Paper.
4. List with q=OLD and no category. Expect The Sunday Room and Ink & Paper.
5. List with q=spaceship and no category. Expect [].
6. Retrieve ID 999999, after confirming it is absent. Expect 404. Invalid non-integer IDs must return 422; use the terminal if /docs blocks that input.

Windows PowerShell

```
curl.exe -i "$base/api/places"
$places = Invoke-RestMethod "$base/api/places"
$existingId = $places[0].id
curl.exe -i "$base/api/places/$existingId"
curl.exe -i "$base/api/places?q=old&category=Little%20shops"
curl.exe -i "$base/api/places?q=OLD"
curl.exe -i "$base/api/places?q=spaceship"
curl.exe -i "$base/api/places/999999"
curl.exe -i "$base/api/places/abc"
```

[FastAPI: query parameters](#)

[FastAPI: handling errors](#)

Check 2: Create a recommendation

FastAPI /docs first

1. Open POST /api/places and submit the following complete body.
2. Expect 201, the same content, an integer id, and shortlisted=false. Note the ID for the following checks.
3. Retrieve that ID with GET /api/places/{place_id}. Expect 200 and the same record.

```
{
  "name": "Practice Corner",
  "category": "Little shops",
  "neighborhood": "Old Town",
  "address": "14 Practice Lane",
  "description": "A small shop used to check the local places guide during this project.",
  "tip": "Look for the notebooks near the entrance."
}
```

Windows PowerShell

```
$placeBody = @{
  name = 'Practice Corner'
  category = 'Little shops'
  neighborhood = 'Old Town'
  address = '14 Practice Lane'
  description = 'A small shop used to check the local places guide during this project.'
  tip = 'Look for the notebooks near the entrance.'
}
$created = Invoke-RestMethod -Method Post `
  -Uri "$base/api/places" -ContentType 'application/json' `
  -Body ($placeBody | ConvertTo-Json)
$placeId = $created.id
$created | ConvertTo-Json
curl.exe -i "$base/api/places/$placeId"
```

Invoke-RestMethod displays the returned data. Check the explicit POST status in /docs, or in the automated creation test. Do not submit id or shortlisted in this body.

[FastAPI: request bodies and validation models](#)

Check 3: Reject invalid requests

FastAPI /docs first

1. Note the number of places returned by GET /api/places.
2. In POST, start with the page 6 body. Change name to one space. Expect 422. Restore the body before each following case.
3. Remove address, set category to Other, or add an extra field price. Each request must return 422.
4. For each text field, test one character below its minimum and one above its maximum. Expect 422. Values at the limits are accepted when the remaining body is valid.
5. Set name to " Practice Corner ". Expect acceptance with the returned name "Practice Corner". Delete this extra example after checking it.
6. Confirm rejected POST requests did not add records. Query category=Other and q with 101 characters must also return 422.

Windows PowerShell: representative invalid requests

This writes a temporary request file in your project folder. curl.exe reads the body from the file to avoid JSON quoting differences between Windows shells.

```
$before = @(Invoke-RestMethod "$base/api/places").Count
$invalid = $placeBody.Clone()
$invalid.name = ''
$invalid | ConvertTo-Json | Set-Content invalid-request.json -Encoding UTF8
curl.exe -i -X POST "$base/api/places" `
  -H "Content-Type: application/json" `
  --data-binary "@invalid-request.json"
$after = @(Invoke-RestMethod "$base/api/places").Count
"Before: $before; after: $after"
curl.exe -i "$base/api/places?category=Other"
$longQuery = 'x' * 101
curl.exe -i "$base/api/places?q=$longQuery"
```

Expect 422 for all three invalid HTTP requests and equal before/after counts. Repeat the file-based POST with each body variation above. /docs may prevent an invalid query value locally; the terminal sends it directly to the server.

422 responses describe which field failed validation. Their complete text need not be identical across dependency versions. Invalid changes to an existing place must leave its stored content unchanged.

[FastAPI: request body validation](#)

Check 4: Replace a recommendation

FastAPI /docs first

1. Use the ID created on page 6. In PUT /api/places/{place_id}, send all six fields with name changed to Updated Practice Corner.
2. Expect 200, the same ID, and the new name. GET that ID to confirm the change was stored.
3. Send only {"name":"Updated Practice Corner"}. Expect 422 because the remaining required fields are missing.
4. Send the complete valid body to absent ID 999999. Expect 404, not a new record.

Windows PowerShell

```
$placeBody.name = 'Updated Practice Corner'
Invoke-RestMethod -Method Put -Uri "$base/api/places/$placeld" `
  -ContentType 'application/json' `
  -Body ($placeBody | ConvertTo-Json)
curl.exe -i "$base/api/places/$placeld"
@{ name = 'Updated Practice Corner' } | ConvertTo-Json | `
  Set-Content invalid-request.json -Encoding UTF8
curl.exe -i -X PUT "$base/api/places/$placeld" `
  -H "Content-Type: application/json" `
  --data-binary "@invalid-request.json"
$placeBody | ConvertTo-Json | `
  Set-Content valid-request.json -Encoding UTF8
curl.exe -i -X PUT "$base/api/places/999999" `
  -H "Content-Type: application/json" `
  --data-binary "@valid-request.json"
```

PUT replaces editable content. It does not assign a new ID or change shortlist membership. Check the membership rule after saving the place on the next page.

[FastAPI: request bodies](#)

[FastAPI: missing-record errors](#)

Check 5: Shared shortlist

FastAPI /docs first

1. PUT /api/places/{place_id}/shortlist for your example with {"shortlisted":true}. Expect 200 and shortlisted=true.
2. Execute the same request again. GET /api/shortlist must contain this ID exactly once.
3. Repeat the complete content PUT from page 8. The response must still show shortlisted=true.
4. Set shortlisted=false twice. Both requests succeed. The record leaves the shortlist but GET by ID still returns 200.
5. Submit {"shortlisted":"true"}, {"shortlisted":1}, or an extra field. Expect 422 and no membership change. Saving missing ID 999999 with a valid boolean returns 404.

Windows PowerShell

```
$save = @{ shortlisted = $true } | ConvertTo-Json
$unsave = @{ shortlisted = $false } | ConvertTo-Json
$uri = "$base/api/places/$placeld/shortlist"
Invoke-RestMethod -Method Put -Uri $uri `
  -ContentType 'application/json' -Body $save
Invoke-RestMethod -Method Put -Uri $uri `
  -ContentType 'application/json' -Body $save
Invoke-RestMethod "$base/api/shortlist" | ConvertTo-Json -Depth 5
Invoke-RestMethod -Method Put -Uri "$base/api/places/$placeld" `
  -ContentType 'application/json' `
  -Body ($placeBody | ConvertTo-Json)
Invoke-RestMethod -Method Put -Uri $uri `
  -ContentType 'application/json' -Body $unsave
Invoke-RestMethod -Method Put -Uri $uri `
  -ContentType 'application/json' -Body $unsave
curl.exe -i "$base/api/places/$placeld"
```

To check invalid shortlist bodies in the terminal, write each variation to invalid-request.json as on page 7, then use curl.exe -i -X PUT with \$uri and --data-binary "@invalid-request.json". Use the same request with a valid boolean and missing ID to check 404.

[Python: SQLite operations and constraints](#)

Check 6: Delete a saved place

FastAPI /docs first

1. Save your example again using {"shortlisted":true}. Confirm its ID appears in GET /api/shortlist.
2. DELETE /api/places/{place_id}. Expect 204 with no response body.
3. GET the deleted ID and DELETE it again. Both return 404.
4. GET /api/shortlist. The deleted ID must be absent. Other saved places must remain.

Windows PowerShell

```
Invoke-RestMethod -Method Put `
  -Uri "$base/api/places/$placeId/shortlist" `
  -ContentType 'application/json' -Body $save
curl.exe -i -X DELETE "$base/api/places/$placeId"
curl.exe -i "$base/api/places/$placeId"
curl.exe -i -X DELETE "$base/api/places/$placeId"
curl.exe -i "$base/api/shortlist"
```

Database relationship checks: terminal only

The check_shortlist script must create its own temporary place, save it, read it through a join, confirm that duplicate membership is rejected, delete the place, confirm membership was removed, and confirm that a missing place cannot be saved. Clean up the temporary place even if a check fails.

```
.\.venv\Scripts\python.exe -m scripts.check_shortlist
```

Expect the five relationship checks to succeed. The script may catch expected SQLite integrity errors, but must fail if a constraint accepts invalid data. These are database checks; /docs cannot execute raw SQL.

[Python: SQLite and integrity errors](#)

Check 7: Persistence and initialization

FastAPI /docs first, with terminal restarts

1. POST a new example using the page 6 body, record its ID, then save it to the shortlist.
2. Stop and restart the server in terminal A. GET the new ID and the shortlist. The place and membership must remain.
3. Edit its name, restart, and GET it again. The edited name must remain.
4. Delete that example, restart, and GET its ID. Expect 404; its shortlist entry must also remain absent.

Windows PowerShell: create and save

```
$created = Invoke-RestMethod -Method Post -Uri "$base/api/places" `
    -ContentType 'application/json' `
    -Body ($placeBody | ConvertTo-Json)
$placeld = $created.id
Invoke-RestMethod -Method Put `
    -Uri "$base/api/places/$placeld/shortlist" `
    -ContentType 'application/json' -Body $save
```

Restart in terminal A, keeping terminal B open so its variables remain available. In terminal B:

```
curl.exe -i "$base/api/places/$placeld"
curl.exe -i "$base/api/shortlist"
```

Repeat the page 8 update commands, restart and GET. Then repeat the page 10 delete commands, restart and GET. Confirm the same results as the browser sequence.

Seed-once check: terminal only

```
.\.venv\Scripts\python.exe -m app.db
.\.venv\Scripts\python.exe -m scripts.inspect_guide_database
.\.venv\Scripts\python.exe -m app.db
.\.venv\Scripts\python.exe -m scripts.inspect_guide_database
```

Both inspections must show the same records and Seed marker: 1. Changing seed.json after initialization must not overwrite stored data. Also verify, using the temporary-database test on page 12, that deleting every place and restarting leaves an empty collection rather than reseeding.

[Python: SQLite transactions and connections](#)

Automated tests and frontend check

Automated tests: terminal only

An automated test sends requests or runs operations, then checks the results with assertions. A failing assertion means the observed result differs from the expected result. Use temporary databases so tests do not change the demonstration data.

Create fixtures in `tests/conftest.py` that redirect the database path before starting `TestClient`. Use its context manager so application startup runs. Cover these five groups in `tests/test_api.py`:

1. Listing, combined filters, and missing-record responses.
2. Rejected request bodies, query limits, and strict shortlist booleans without unintended writes.
3. Create, get, replace, save twice, unsave twice, and delete. Check stable identity, membership, cascade cleanup, and status codes.
4. Restart with the same temporary database. Keep edits and membership, and do not restore deleted data. Delete every place and restart to prove the seed marker prevents reseeding.
5. Direct database writes reject duplicate membership and references to missing places.

```
..\env\Scripts\python.exe -m pytest -q
```

Expect all tests to pass. Uvicorn does not need to run for `TestClient` tests. A test must fail when the behavior it checks is broken.

[FastAPI: testing with TestClient](#)

Frontend: final integration requirement

Open `http://127.0.0.1:8000/`. Browse and filter places, create an example, edit it, save and unsave it, then delete it. Refresh the page after changes and confirm the displayed data matches the API. The browser page must use the API rather than a separate collection in JavaScript.

These checks use the application page, not `/docs`. A terminal can verify that files are served, but cannot prove that buttons and forms work:

[FastAPI: serving static files](#)